Лабораторна робота № 3

Розв’язання комбінаторної задачі розміщення з обмеженнями з використанням генетичного алгоритму та алгоритму імітації відпалу

Виконав: Олійник Гліб, група ІПЗ-24-2, Варіант № 25

Мета роботи: вивчення принципів роботи генетичних алгоритмів, набуття навичок самостійного кодування розв’язку у вигляді хромосоми та побудови фітнес-функції для задачі з обмеженнями, а також порівняння ефективності генетичного алгоритму й алгоритму імітації відпалу на одній і тій самій задачі.

Завдання до лабораторної роботи:

Завдання 1. Розв’язання задачі генетичним алгоритмом:

1. Реалізувати програму, яка зчитує варіант із JSON-файлу і перевіряє коректність вхідних даних (рифи в межах поля, кораблі не довші за поле).

2. Самостійно розробити кодування розстановки у вигляді хромосоми та обґрунтувати вибір.

3. Самостійно розробити фітнес-функцію та обґрунтувати її.

4. Реалізувати генетичні оператори: відбір, схрещування, мутацію.

5. Реалізувати візуалізацію поля з розстановкою кораблів і графік найкращого та середнього значення фітнес-функції за поколіннями.

6. Дослідити вплив розміру популяції M та ймовірності мутації pm на результат.

Завдання 2. Розв’язання задачі алгоритмом імітації відпалу:

1. Реалізувати розв'язання тієї самої задачі алгоритмом імітації відпалу (SA).

2. Визначити й обґрунтувати подання стану, функцію енергії E, спосіб генерації сусіднього стану, Tstart, закон охолодження і критерій зупинки.

3. Реалізувати візуалізацію поля та графіки поточної й найкращої енергії, температури T і частки прийнятих переходів.

4. Дослідити вплив швидкості охолодження α на результат.

5. Провести порівняльне дослідження ГА та SA на одному варіанті.

6. Порівняти поведінку обох алгоритмів на складнішому варіанті.

Хід роботи

1. Постановка задачі та вхідні дані

Задача полягає в розміщенні флоту кораблів гри “Морський бій” на квадратному полі N x N з підводними рифами. Координати клітинки (x, y): x - номер стовпця зліва направо, y - номер рядка згори вниз, нумерація від 0. Кожен корабель - прямолінійний відрізок із l клітинок. Розстановка правильна, якщо виконуються всі правила:

1. Кожен корабель розташований горизонтально або вертикально.

2. Кожен корабель повністю розташований у межах поля.

3. Жодна клітинка корабля не розташована на рифі.

4. Кораблі не перетинаються.

5. Кораблі не торкаються один одного ні стороною, ні кутом (торкатися рифів дозволено).

Варіант 25 за списком відповідає варіанту 5, тому використано файл variant05.json. Поле має розмір 9 x 9, флот складається з 10 кораблів довжин 5, 4, 3, 3, 2, 2, 1, 1, 1, 1 (23 клітинки), на полі 40 рифів. Із 81 клітинки вільні 41, і кораблі займають більше половини з них.

Вміст файлу variant05.json:

```
{
    "field_size": 9,
    "ships": [5, 4, 3, 3, 2, 2, 1, 1, 1, 1],
    "reefs": [
        [0, 1], [0, 4], [0, 5], [1, 1], [1, 3], [1, 5], [1, 6],
        [1, 8], [2, 4], [2, 7], [2, 8], [3, 2], [3, 4], [3, 5],
        [3, 7], [4, 2], [4, 4], [4, 5], [4, 7], [5, 0], [5, 2],
        [5, 3], [5, 4], [5, 7], [6, 0], [6, 1], [6, 4], [6, 5],
        [6, 7], [7, 0], [7, 1], [7, 3], [7, 4], [7, 5], [7, 6],
```

```
            [7, 7], [8, 0], [8, 4], [8, 5], [8, 8]
        ]
    }
```

Поле варіанта з рифами показано на Рис. 1 (коричневі клітинки - рифи, блакитні - вода).

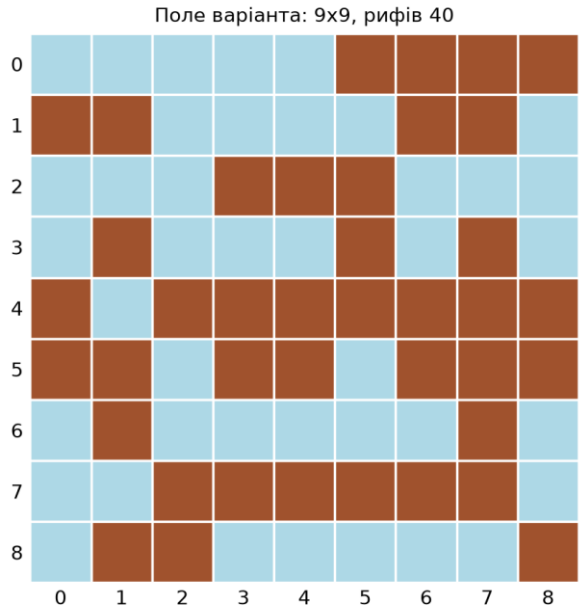


Рис. 1. Поле варіанта 5 з рифами

Варіант зчитує функція load_variant(), а коректність даних перевіряє check_data(): наявність полів field_size, ships, reefs, додатний розмір поля, довжини кораблів (не більші за поле), координати рифів у межах поля без повторів, а також наявність допустимих положень для кожної довжини корабля. Усі знайдені помилки виводяться разом (Табл. 1).

Таблиця 1. Приклади повідомлень про помилки вхідних даних

Некоректні дані	Повідомлення програми
“ships”: [4] при N = 3	Корабель №1: довжина 4 більша за розмір поля 3.
“reefs”: [[5, 0]] при N = 5	Риф №1: (5, 0) поза межами поля 5x5.
відсутнє поле reefs	Відсутнє поле “reefs”.
“field_size”: true	field_size має бути додатним цілим числом, отримано True.
корабель 3 на полі 3 x 3, де всі ряди перекрито рифами	Для кораблів довжини 3 немає жодного допустимого положення.

2. Кодування розстановки у вигляді хромосоми

Обрано цілочисельне кодування. Перед запуском для кожної довжини корабля один раз будується список допустимих положень - трійок (x, y, орієнтація) першої клітинки, за яких корабель лежить у межах поля і не стоїть на рифах. Для кораблів довжини 1 орієнтація одна.

Ген - номер положення і-го корабля у списку положень його довжини: gi = 0, 1, …, n(l) - 1, де n(l) - кількість допустимих положень корабля довжини l. Хромосома - список генів усіх кораблів у порядку файлу, тому її довжина L = 10. Ген за списком перетворюється на положення (x, y, орієнтація), з якого обчислюються клітинки корабля. Кожна хромосома однозначно декодується в розстановку, а будь-яка розстановка, що задовольняє правила 1-3, має своє кодування, тому шуканий розв'язок у цьому кодуванні є.

Таблиця 2. Кількість допустимих положень кораблів варіанта

Довжина корабля	Кількість кораблів	Допустимих положень n(L)
5	1	3
4	1	8
3	2	18
2	2	36
1	4	41

Простір пошуку містить 3 * 8 * 18^2 * 36^2 * 41^4 = 2,85 * 10^13 хромосом, тому повний перебір неможливий.

Автоматично виконуються правило 1 (положення задається першою клітинкою й орієнтацією) та правила 2 і 3 (у список потрапляють лише положення в межах поля і без рифів). Правила 4 (перетин) і 5 (дотик) автоматично не виконуються, їх враховує фітнес-функція.

Для кожного положення також запам’ятовуються сусідні положення того самого корабля (перша клітинка зміщена не більше ніж на одну клітинку, зокрема поворот на місці). Вони використовуються в мутації: мала зміна гена означає малий зсув корабля, як і вимагається для кодування.

Лістинг коду:

```
def build_placements(self, length):
    ids = []
    orientations = [HORIZONTAL, VERTICAL] if length > 1 else [HORIZONTAL]
    for orientation in orientations:
        for y in range(self.size):
            for x in range(self.size):
                cells = ship_cells(length, x, y, orientation)
                if all(self.inside(a, b) and (a, b) not in self.reefs
                       for a, b in cells):
                    ids.append(len(self.shapes))
```

```
                        self.shapes.append((length, x, y, orientation))
                        self.cells.append(cells)
        self.placements[length] = ids
    def placement_id(self, index, gene):
        return self.placements[self.ships[index]][gene]


    def decode(self, genes):
        return [self.shapes[self.placement_id(i, gene)] for i, gene in
    enumerate(genes)]
```

3. Фітнес-функція

Фітнес-функцію побудовано як штрафну функцію: F(x) = f(x) - Σ wi * Pi(x), де f(x) = 0, бо іншої мети, крім виконання правил, у задачі немає:

$$F = -(10 * Pout + 10 * Preef + 4 * Poverlap + 1 * Ptouch)$$

Тут Ci - множина клітинок і-го корабля, а міри порушень обчислюються по всіх парах кораблів i < j:

- Pout - кількість клітинок кораблів поза полем;

- Preef - кількість клітинок кораблів на рифах;

- Poverlap = Σ |Ci ∩ Cj| - кількість спільних клітинок кожної пари кораблів;

- Ptouch - кількість пар клітинок a з Ci і b з Cj, a != b, для яких max(|xa – xb|, |ya – yb|) = 1, тобто клітинки сусідні стороною або кутом.

Таблиця 3. Врахування правил задачі

Правило	Як враховано
1. Горизонтально або вертикально	кодуванням: положення - перша клітинка й орієнтація
2. У межах поля	кодуванням; у формулі - доданок Pout (завжди 0)
3. Не на рифі	кодуванням; у формулі - доданок Preef (завжди 0)
4. Не перетинаються	штраф 4 за кожну спільну клітинку (Poverlap)
5. Не торкаються	штраф 1 за кожну пару сусідніх клітинок різних кораблів (Ptouch)

Pout і Preef для декодованих хромосом завжди дорівнюють 0, але їх залишено у формулі, щоб фітнес відповідав усім п’яти правилам і міг оцінювати будь-яку розстановку. Перетин має більшу вагу, ніж дотик: щоб його усунути, корабель треба відсунути щонайменше на дві клітинки, а дотик часто виправляється зсувом на одну.

Розрізнення неправильних розстановок. Фітнес рахує не лише факт порушення, а його обсяг, тому розстановка, ближча до правильної, має більший F.

У Табл. 4 наведено фітнес правильної розстановки, двох неправильних (зсув одного однопалубного корабля) та середнє значення для випадкових хромосом.

Таблиця 4. Приклади значень фітнес-функції

Розстановка	overlap	touch	F
правильна	0	0	0
корабель 10 біля корабля 2 (дотик)	0	1	-1
корабель 10 на кораблі 2 (перетин)	1	2	-6
випадкова хромосома (середнє з 2000)	-	-	-49,9

Значення фітнесу правильної розстановки. Усі доданки невід’ємні, тому F <= 0, а F = 0 тоді й лише тоді, коли не порушено жодного правила. Оптимум відомий заздалегідь, тому критерієм зупинки є досягнення F = 0. Кількість перетинів і дотиків кожної пари положень обчислюється один раз і запам'ятовується.

Лістинг коду:

```
def pair_counts(self, first, second):
    key = (first, second) if first < second else (second, first)
    if key in self.pair_cache:
        return self.pair_cache[key]
    overlap = 0
    touch = 0
    for a in self.cells[first]:
        for b in self.cells[second]:
            if a == b:
                overlap += 1
            elif abs(a[0] - b[0]) <= 1 and abs(a[1] - b[1]) <= 1:
                touch += 1
    self.pair_cache[key] = (overlap, touch)
    return overlap, touch


def counts(self, genes):
    ids = [self.placement_id(i, gene) for i, gene in enumerate(genes)]
    overlap = 0
    touch = 0
    for i in range(len(ids)):
        for j in range(i + 1, len(ids)):
            o, t = self.pair_counts(ids[i], ids[j])
            overlap += o
            touch += t
    return {"out_of_field": 0, "on_reef": 0, "overlap": overlap, "touch": touch}
```

```
def fitness_from_counts(counts):
    return -(W_OUT * counts["out_of_field"] + W_REEF * counts["on_reef"]
             + W_OVERLAP * counts["overlap"] + W_TOUCH * counts["touch"])
```

4. Генетичні оператори та параметри ГА

Ініціалізація: популяція з M хромосом, кожен ген - випадковий номер допустимого положення відповідного корабля.

Відбір. Основний - турнірний: з популяції випадково вибираються k = 3 особини, переможець - найкраща. Також реалізовано відбір рулеткою pi = Fi / Σ Fj; оскільки F <= 0, фітнес зсувається: Fi’ = Fi - Fmin + 1.

Схрещування з імовірністю pc = 0,9: двоточкове (основне) або одноточкове. Ген з номером і в обох батьків описує той самий корабель, тому обмін ділянками дає хромосоми, що коректно декодуються. З імовірністю 1 - pc нащадки є копіями батьків.

Мутація: кожен ген з імовірністю pm = 1 / L = 0,1 змінюється. З імовірністю 0,5 його замінюють на сусіднє положення (зсув на одну клітинку або поворот), інакше - на випадкове інше. Перше дає малі зміни біля розв'язку, друге підтримує різноманітність популяції.

Елітизм: дві найкращі особини без змін переходять у нове покоління.

Критерії зупинки: F = 0; досягнуто max_gen = 1000 поколінь; найкращий F не покращувався stall = 300 поколінь. Також реалізовано перезапуск популяції при стагнації (за замовчуванням вимкнений).

Таблиця 5. Параметри генетичного алгоритму

Параметр	Значення	Призначення
M	100	розмір популяції
pc	0,9	імовірність схрещування
pm	0,1	імовірність мутації гена (1/L)
k	3	розмір турніру
elite	2	кількість елітних особин
max_gen	1000	найбільша кількість поколінь
stall	300	поколінь без покращення до зупинки
відбір	турнірний	або рулетка
схрещування	двоточкове	або одноточкове

Лістинг коду:

```
def tournament(self):
```

```
    size = max(1, min(self.params.tournament, len(self.population)))
    chosen = self.random.sample(range(len(self.population)), size)
    winner = max(chosen, key=lambda i: self.scores[i])
    return self.population[winner]


def crossover(self, first, second):
    if self.length < 2 or self.random.random() >= self.params.pc:
        return list(first), list(second)
    if self.params.crossover == ONE_POINT or self.length < 3:
        cut = self.random.randint(1, self.length - 1)
        return first[:cut] + second[cut:], second[:cut] + first[cut:]
    left, right = sorted(self.random.sample(range(1, self.length), 2))
    child_a = first[:left] + second[left:right] + first[right:]
    child_b = second[:left] + first[left:right] + second[right:]
    return child_a, child_b
```

Лістинг коду:

```
def mutate(self, genes):
    for index in range(self.length):
        if self.random.random() >= self.params.pm:
            continue
        genes[index] = self.new_gene(index, genes[index])
    return genes


def new_gene(self, index, gene):
    length = self.variant.ships[index]
    close = self.variant.neighbours[length][gene]
    if close and self.random.random() < self.params.local_share:
        return self.random.choice(close)
    count = self.variant.gene_range(index)
    if count == 1:
        return gene
    value = self.random.randrange(count - 1)
    return value if value < gene else value + 1
```

5. Псевдокод генетичного алгоритму

Псевдокод:

```
Початок:
    зчитати variant.json і перевірити вхідні дані
    для кожної довжини корабля побудувати список допустимих положень
    P = M випадкових хромосом, обчислити F кожної особини
Повторювати:
```

```
    1. Запам'ятати найкращий і середній F покоління.
    2. Якщо найкращий F = 0 - розв’язок знайдено, завершити.
    3. Якщо покоління = max_gen - завершити (розв'язок не знайдено).
    4. Якщо найкращий F не покращувався stall поколінь:
           якщо увімкнено перезапуск - P = M нових випадкових хромосом,
           інакше завершити (розв'язок не знайдено).
    5. Q = elite найкращих особин P.
    6. Поки |Q| < M:
           a = відбір(P), b = відбір(P)
           з імовірністю pc: (c1, c2) = схрещування(a, b),
           інакше c1 = копія a, c2 = копія b
           кожен ген c1 і c2 з імовірністю pm змінити (мутація)
           обчислити F(c1), F(c2) і додати їх до Q
    7. P = Q, покоління = покоління + 1.
Вивести найкращу розстановку, її фітнес і перелік порушених правил.
```

6. Візуалізація та результат роботи ГА

Головне вікно має вкладки “Генетичний алгоритм” та “Імітація відпалу”. Ліворуч - параметри і текстовий результат, праворуч - поле і графіки. На полі блакитні клітинки - вода, коричневі - рифи, темно-сині - кораблі з номерами, а клітинки, що порушують правила (перетин або дотик), виділяються червоним. Над полем виводиться фітнес і кількість порушень кожного типу. Праворуч будується графік найкращого (суцільна лінія) та середнього (штрихова) фітнесу за поколіннями (Рис. 2).

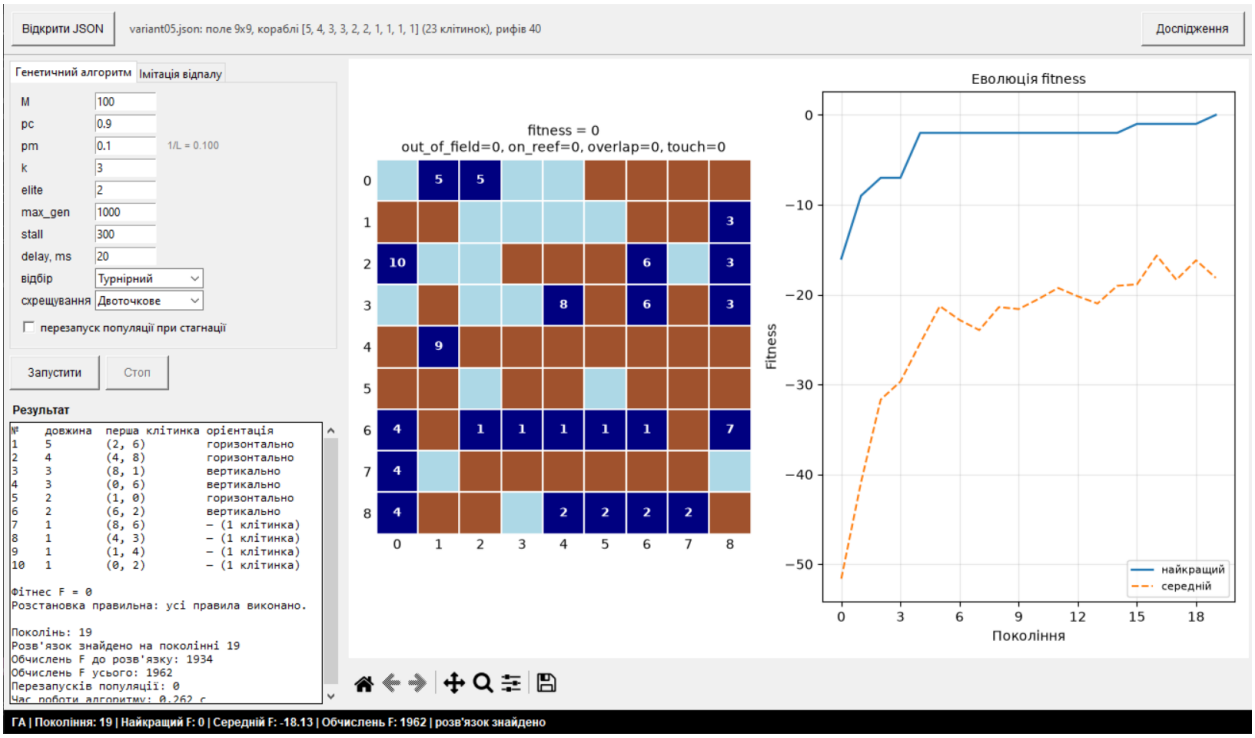


Рис. 2. Вікно програми після розв'язання задачі генетичним алгоритмом

Правильну розстановку знайдено на поколінні 19, на 1934-му обчисленні фітнес-функції (час роботи 0,262 с). Найкращий фітнес початкової популяції становив -16, середній - -51,6; до знаходження розв'язку середній фітнес зріс до -18,1. Розстановку наведено в Табл. 6.

Таблиця 6. Розстановка, знайдена генетичним алгоритмом (F = 0)

№	Довжина	Перша клітинка (x, y)	Орієнтація
1	5	(2, 6)	горизонтально
2	4	(4, 8)	горизонтально
3	3	(8, 1)	вертикально
4	3	(0, 6)	вертикально
5	2	(1, 0)	горизонтально
6	2	(6, 2)	вертикально
7	1	(8, 6)	- (1 клітинка)
8	1	(4, 3)	- (1 клітинка)
9	1	(1, 4)	- (1 клітинка)
10	1	(0, 2)	- (1 клітинка)

Якщо правильну розстановку не знайдено, програма виводить найкращу з знайдених і перелік порушених правил. Приклад при max_gen = 3:

```
Фітнес F = -5
Правильну розстановку не знайдено. Порушені правила:
  5. Кораблі торкаються стороною або кутом - 5
```

7. Дослідження впливу параметрів ГА

Для кожного значення параметра виконано 10 запусків (seed = 0…9), решта параметрів - з Табл. 5. Фіксувалися частка успішних запусків, середня кількість поколінь і обчислень фітнес-функції до знаходження розв'язку (за успішними запусками) та середній час. Результати для розміру популяції M наведено в Табл. 7 і на Рис. 3.

Таблиця 7. Вплив розміру популяції M

M	Успішних запусків	Частка успішних, %	Сер. покоління	Сер. обчислень F	Сер. час, с
20	9/10	90	43,9	797	0,042
50	10/10	100	21,3	1048	0,037
100	10/10	100	18,6	1876	0,062
200	10/10	100	14,0	2871	0,099

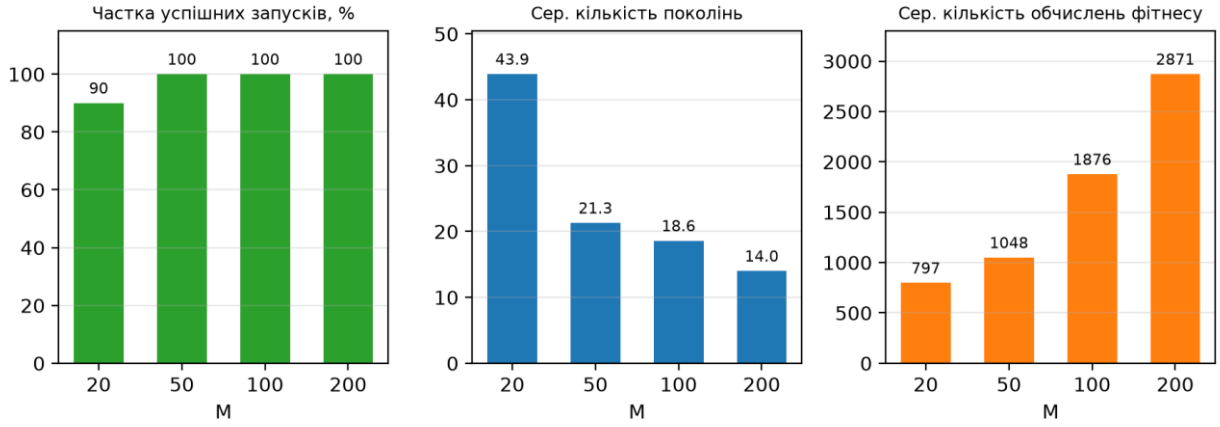


Рис. 3. Вплив розміру популяції M на роботу ГА

Мала популяція M = 20 найекономніша за кількістю обчислень (797), але найменш надійна: успішними були 9 запусків із 10, бо в малій популяції швидко втрачається різноманітність. Зі збільшенням M кількість поколінь зменшується (з 43,9 до 14,0), але кожне покоління коштує M обчислень, тому загальна кількість обчислень зростає до 2871 при M = 200. Оптимальними для цього варіанта є M = 50…100.

Результати для ймовірності мутації pm наведено в Табл. 8 і на Рис. 4.

Таблиця 8. Вплив ймовірності мутації pm

pm	Успішних запусків	Частка успішних, %	Сер. покоління	Сер. обчислень F	Сер. час, с
0,02	10/10	100	13,2	1352	0,043
0,1	10/10	100	18,6	1876	0,062
0,2	9/10	90	37,2	3703	0,221
0,4	5/10	50	166,4	16355	1,070

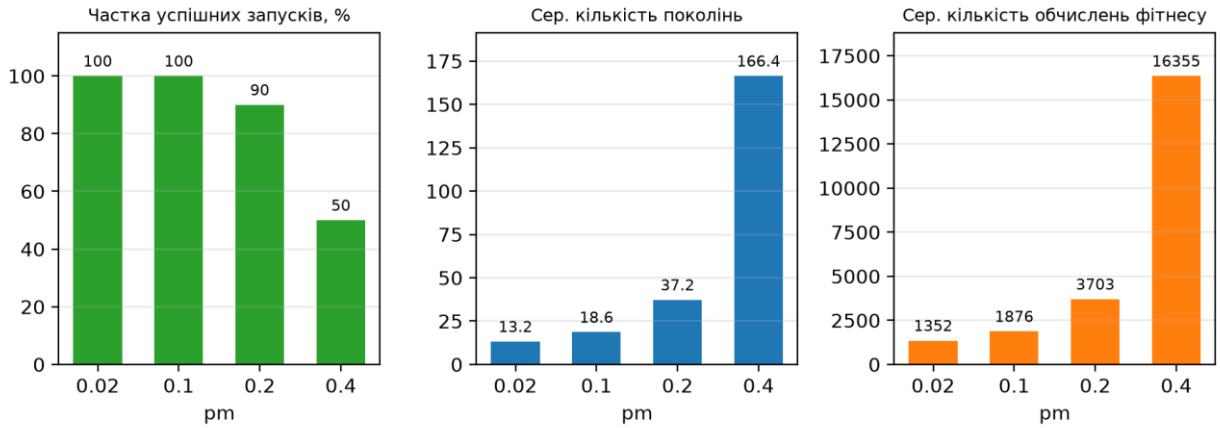


Рис. 4. Вплив ймовірності мутації pm на роботу ГА

При pm = 0,02…0,1 алгоритм завжди знаходить розв'язок за 13,2-18,6 поколінь. Зі зростанням pm результати різко погіршуються: при pm = 0,4 успішними були лише 5 запусків із 10, а середня кількість поколінь зросла до 166,4. Занадто часта мутація руйнує хороші розстановки, і пошук наближається до випадкового. Значення pm = 1 / L = 0,1 з методичних вказівок дає надійний результат.

8. Розв’язання задачі алгоритмом імітації відпалу

Подання стану. Стан - та сама хромосома, що й у ГА, тому правила 1-3 виконуються автоматично, а програма працює з тими самими JSON-файлами.

Функція енергії. Фітнес із завдання 1 використано без змін: E = −F >= 0. Мінімум E = 0 відповідає правильній розстановці.

Сусідній стан. Випадково вибирається один корабель і змінюється лише його положення: з імовірністю 0,5 на сусіднє (зсув на одну клітинку або поворот), інакше на випадкове інше. Це той самий оператор, що й мутація одного гена в ГА.

Критерій Метрополіса (з методичних вказівок до лабораторної роботи № 2): перехід із ΔE <= 0 приймається завжди, а погіршення ΔE > 0 - з імовірністю:

$$P = \exp(-\Delta E / T)$$

Початкова температура Tstart = 5. Типове погіршення від переміщення одного корабля становить 1-5 одиниць енергії, тому при T = 5 на першому ступені приймається близько 60 % переходів (у прогоні на Рис. 5 - 0,59). Пошук починається майже як випадкове блукання, а з охолодженням стає жадібним.

Закон охолодження - геометричний:

$$T_{k+1} = \alpha * T_k,\ \alpha = 0{,}95$$

На кожному ступені температури виконується L = 100 спроб переходу. Також реалізовано лінійний ($T_{k+1} = T_0 - k * \Delta T$) і логарифмічний ($T_k = T_0 / \ln(1 + k)$) закони. Критерій зупинки: E = 0; T < Tmin = 0,01; вичерпано 200 000 обчислень енергії.

Таблиця 9. Параметри алгоритму імітації відпалу

Параметр	Значення	Призначення
Tstart	5	початкова температура
Tmin	0,01	температура зупинки
alpha	0,95	коефіцієнт геометричного охолодження
L	100	спроб переходу на одному ступені T
max_eval	200 000	найбільша кількість обчислень E
закон	геометричний	або лінійний, логарифмічний

Лістинг коду:

```
def neighbour(self):
    genes = list(self.genes)
    index = self.random.randrange(self.length)
    length = self.variant.ships[index]
    close = self.variant.neighbours[length][genes[index]]
    if close and self.random.random() < self.params.local_share:
        genes[index] = self.random.choice(close)
        return genes
    count = self.variant.gene_range(index)
    if count > 1:
        value = self.random.randrange(count - 1)
        genes[index] = value if value < genes[index] else value + 1
    return genes
```

Один ступінь температури (фрагмент методу step()):

```
for _ in range(self.params.iterations):
    candidate = self.neighbour()
    energy = self.evaluate(candidate)
    tried += 1
    delta_e = energy - self.energy
    probability = acceptance(delta_e, self.temperature)
    if probability >= 1.0 or self.random.random() < probability:
        self.genes = candidate
        self.energy = energy
        accepted += 1
    if self.energy < self.best_energy:
        self.best_energy = self.energy
```

```
            self.best_genes = list(self.genes)
        if self.best_energy == 0:
            self.solved_evaluations = self.evaluations
            break
```

9. Візуалізація та результат роботи SA

Для SA поле відображається так само, як для ГА, а праворуч будуються три графіки залежно від номера ступеня температури: поточна і найкраща енергія, температура T та частка прийнятих переходів (Рис. 5).

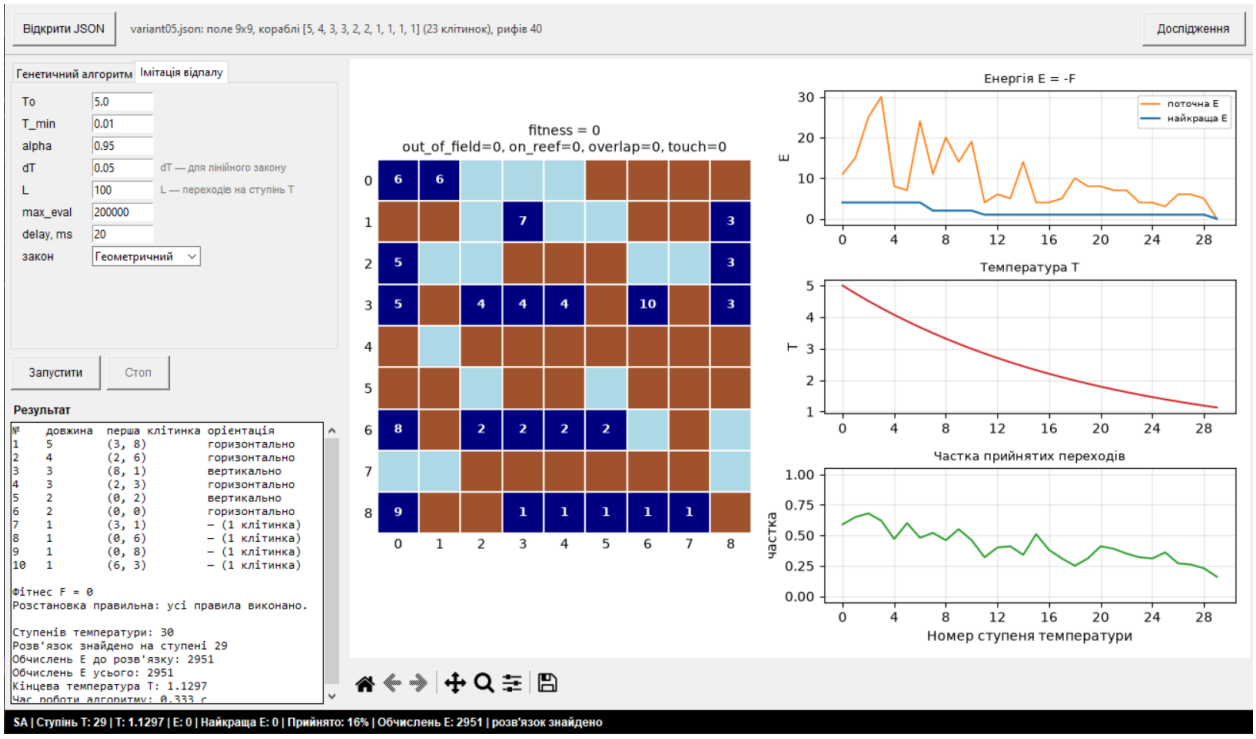


Рис. 5. Вікно програми після розв'язання задачі алгоритмом імітації відпалу

Правильну розстановку знайдено на ступені температури 29, на 2951-му обчисленні енергії (час 0,333 с), коли температура знизилася з 5 до 1,13. Поточна енергія на початку сильно коливається, бо приймається багато погіршень, а частка прийнятих переходів поступово спадає з 0,59 до 0,16. Розстановку наведено в Табл. 10.

Таблиця 10. Розстановка, знайдена алгоритмом імітації відпалу (E = 0)

№	Довжина	Перша клітинка (x, y)	Орієнтація
1	5	(3, 8)	горизонтально
2	4	(2, 6)	горизонтально
3	3	(8, 1)	вертикально
4	3	(2, 3)	горизонтально
5	2	(0, 2)	вертикально
6	2	(0, 0)	горизонтально
7	1	(3, 1)	- (1 клітинка)
8	1	(0, 6)	- (1 клітинка)
9	1	(0, 8)	- (1 клітинка)
10	1	(6, 3)	- (1 клітинка)

10. Дослідження впливу швидкості охолодження α

Для кожного значення α виконано 10 запусків (seed = 0…9), інші параметри - з Табл. 9. Результати наведено в Табл. 11 і на Рис. 6.

Таблиця 11. Вплив швидкості охолодження α

alpha	Успішних запусків	Частка успішних, %	Сер. ступенів T	Сер. обчислень E	Сер. час, с
0,8	10/10	100	10,7	1115	0,026
0,9	10/10	100	15,3	1596	0,039
0,95	10/10	100	26,5	2687	0,063
0,99	10/10	100	81,8	8233	0,188

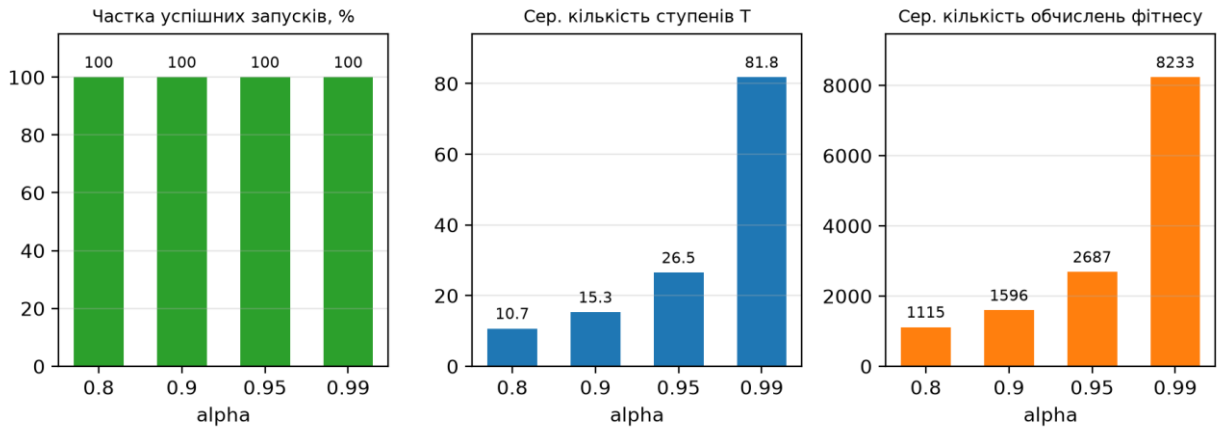


Рис. 6. Вплив швидкості охолодження α на роботу SA

На цьому варіанті всі значення α дали 100 % успішних запусків, а вартість пошуку різна. При α = 0,8 розв’язок знайдено в середньому за 10,7 ступенів і 1115 обчислень, а при α = 0,99 - за 81,8 ступенів і 8233 обчислень. Повільне охолодження довше тримає високу температуру, коли приймається багато погіршень, тому витрачає більше обчислень. Швидке охолодження на складнішому полі може

перетворити SA на жадібний пошук, що застряє в локальному мінімумі, тому для основних запусків обрано α = 0,95.

11. Порівняння ГА та SA

Обидва алгоритми запущено по 10 разів (seed = 0…9) на варіанті 5 з параметрами з Табл. 5 і Табл. 9. Результати наведено в Табл. 12 і на Рис. 7.

Таблиця 12. Порівняння генетичного алгоритму та алгоритму імітації відпалу

Алгоритм	Успішних запусків	Частка успішних, %	Сер. час, с	Сер. обчислень F (E)	Сер. поколінь / ступенів T
ГА	10/10	100	0,068	1876	18,6
SA	10/10	100	0,077	2687	26,5

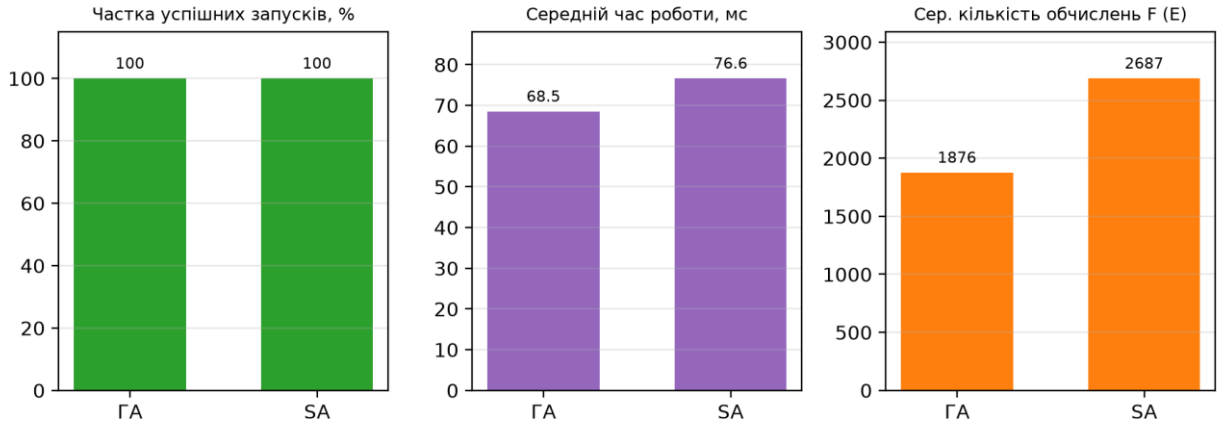


Рис. 7. Порівняння ГА та SA

Обидва алгоритми знайшли розв'язок у всіх запусках за приблизно однаковий час (0,068 с і 0,077 с). ГА знадобилося в середньому 1876 обчислень фітнес-функції, а SA - 2687, тобто в 1,4 раза більше. Час у Табл. 12 виміряно в окремих запусках, тому він трохи відрізняється від Табл. 7 і Табл. 11 для тих самих параметрів.

Чому кількість обчислень фітнес-функції показовіша за кількість поколінь чи ітерацій. Покоління ГА і ступінь температури SA - різні за вартістю кроки: покоління містить M = 100 обчислень фітнесу, а ступінь SA - L = 100 спроб переходу, і при зміні M або L вартість кроку змінюється. Тому порівняння “18,6 поколінь проти 26,5 ступенів” нічого не говорить про обсяг виконаної роботи. Час залежить від комп’ютера, мови реалізації та накладних витрат. Обчислення фітнесу - основна й однакова для обох алгоритмів операція, тому їх кількість до знаходження розв'язку чесно показує, скільки кандидатів переглянув кожен алгоритм.

Висновки: у ході роботи реалізовано програму розв’язання задачі розміщення флоту з обмеженнями генетичним алгоритмом та алгоритмом імітації відпалу. Програма зчитує варіант із JSON-файлу, перевіряє дані, візуалізує поле з розстановкою і графіки та містить вікно дослідження параметрів. Для варіанта 25 (5) обидва алгоритми знаходять правильну розстановку з F = 0 у всіх запусках.

Кодування, у якому ген - номер допустимого положення корабля, автоматично забезпечує правила 1-3 і скорочує простір пошуку. Фітнес у вигляді зваженої суми перетинів і дотиків розрізняє розстановки за обсягом порушень і має відомий оптимум F = 0, який використано як критерій зупинки.

Збільшення M зменшує кількість поколінь, але збільшує кількість обчислень; при M = 20 надійність падає (9 запусків із 10). Імовірність мутації має бути малою (pm = 1 / L): при pm = 0,4 успішними були лише 50 % запусків. Для SA менше α зменшує кількість обчислень (1115 при α = 0,8 проти 8233 при α = 0,99), але на складніших полях підвищує ризик застрягання.

За обраних параметрів (M = 100, α = 0,95) ГА потребував менше обчислень, ніж SA (1876 проти 2687), але при M = 50 (1048) або α = 0,8 (1115) обидва алгоритми працюють швидше, тому результат порівняння залежить від параметрів. Перевага ГА - популяція одночасно досліджує багато розстановок, а схрещування поєднує вдало розміщені групи кораблів; недолік - більше параметрів (M, pc, pm, k, elite) і ризик передчасної збіжності. Перевага SA - простота і мало параметрів (Tstart, α, L), можливість виходити з локальних мінімумів; недолік - сильна залежність від закону охолодження.